

גרף תוכנית (Program Graph)

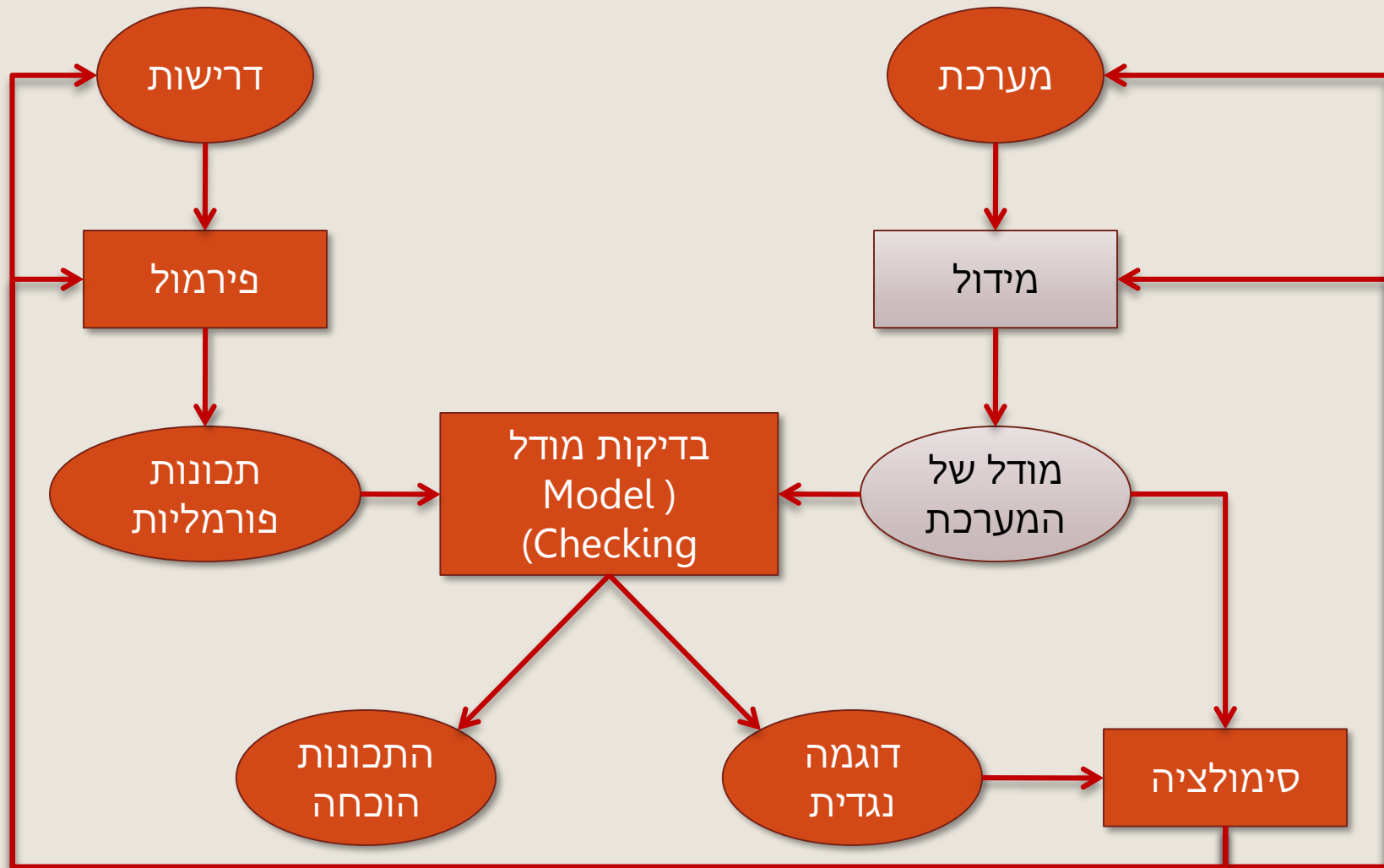
גרא וייס

המחלקה למדעי המחשב

אוניברסיטת בן-גוריון



תזכורת: בדיקות מודל (Model Checking)





מערכת מעברים - Transition System

מערכת מעברים נתונה על-ידי $\langle S, Act, \rightarrow, I, AP, L \rangle$ באשר:

S היא קבוצת המצבים

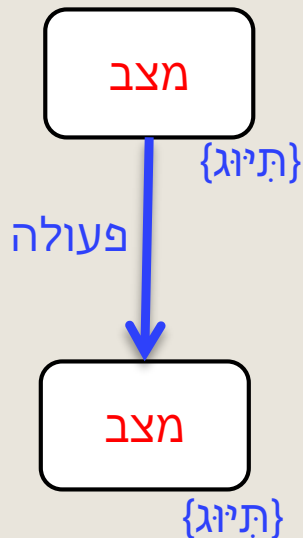
$I \subseteq S$ היא קבוצת המצבים ההתחלתיים

Act היא קבוצת הפעולות

$\rightarrow \subseteq S \times Act \times S$ הוא יחס המעברים

AP היא קבוצת הפסוקים האטומיים

$L: S \rightarrow 2^{AP}$ היא פונקציית תיוג





סמנטיקה: מקטעי ריצות

מקטע ריצה סופי של מערכת מעברים הוא רצף מתחלף של מצבים ופעולות המסתיים במצב:

$$\rho = s_0 \alpha_1 s_1 \alpha_2 \cdots \alpha_n s_n \text{ such that } s_i \xrightarrow{\alpha_{i+1}} s_{i+1} \text{ for all } 0 \leq i < n$$

מקטע ריצה אינסופי של מערכת מעברים הוא רצף מתחלף אינסופי של מצבים ופעולות:

$$\rho = s_0 \alpha_1 s_1 \alpha_2 \cdots \text{ such that } s_i \xrightarrow{\alpha_{i+1}} s_{i+1} \text{ for all } 0 \leq i$$

כדי להקל בקריאה נשתמש גם בסימון $\rho = s_0 \xrightarrow{\alpha_1} s_1 \xrightarrow{\alpha_2} s_2 \xrightarrow{\alpha_3} \cdots \xrightarrow{\alpha_n} s_n$



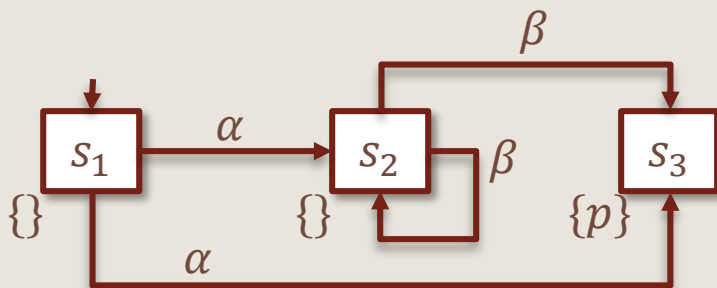
סמנטיקה: ריצות ומקטעי ריצות

מקטע ריצה $s_0 \xrightarrow{\alpha_1} s_1 \xrightarrow{\alpha_2} s_2 \xrightarrow{\alpha_3} \dots \xrightarrow{\alpha_n} s_n$ הוא התחלתי אם $s_0 \in I$

מקטע ריצה מקסימאלי הוא:

מקטע ריצה סופי המסתיים במצב סופני (ללא עוקבים)
או
מקטע ריצה אינסופי

ריצה של מערכת מעברים היא מקטע ריצה התחלתי ומקסימאלי



$$s_1 \xrightarrow{\alpha} \left(s_2 \xrightarrow{\beta} \right)^\omega$$

$$s_1 \xrightarrow{\alpha} s_2 \xrightarrow{\beta} s_2 \xrightarrow{\beta} s_3$$

$$s_1 \xrightarrow{\alpha} s_2 \xrightarrow{\beta} s_2$$

$$s_1 \xrightarrow{\alpha} s_2$$



דוגמאות ריצה

$$\rho_1 = \text{pay} \xrightarrow{\text{coin}} \text{select} \xrightarrow{\tau} \text{sprite} \xrightarrow{\text{get_sprite}} \text{pay} \xrightarrow{\text{coin}} \text{select} \xrightarrow{\tau} \text{sprite} \xrightarrow{\text{get_sprite}} \dots$$

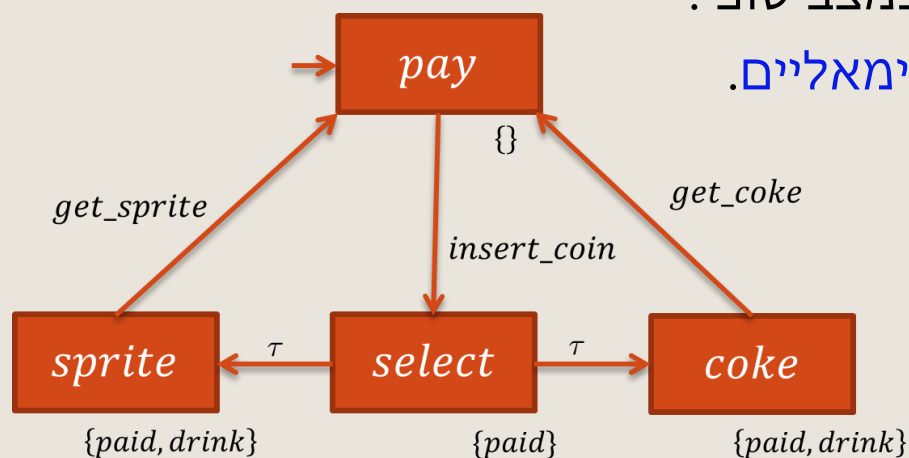
$$\rho_2 = \text{select} \xrightarrow{\tau} \text{sprite} \xrightarrow{\text{get_sprite}} \text{pay} \xrightarrow{\text{coin}} \text{select} \xrightarrow{\tau} \text{coke} \xrightarrow{\text{get_coke}} \dots$$

$$\rho_3 = \text{pay} \xrightarrow{\text{coin}} \text{select} \xrightarrow{\tau} \text{sprite} \xrightarrow{\text{get_sprite}} \text{pay} \xrightarrow{\text{coin}} \text{select} \xrightarrow{\tau} \text{sprite}$$

מקטעי הריצה ρ_1 ו- ρ_3 הם התחלתיים. ρ_2 אינו התחלתי.

ρ_3 אינו מקסימאלי כיוון שאינו מסתיים במצב סופי.

בהנחה ש- ρ_1 ו- ρ_2 אינסופיים, הם מקסימאליים.





מצבים נגישים (Reachable States)

- מצב $s \in S$ נקרא **נגיש** אם קיים מקטע ריצה התחלתי המסתיים ב s :

$$s_0 \xrightarrow{\alpha_1} s_1 \xrightarrow{\alpha_2} s_2 \xrightarrow{\alpha_3} s_3 \xrightarrow{\alpha_4} \dots \xrightarrow{\alpha_n} s_n = s$$

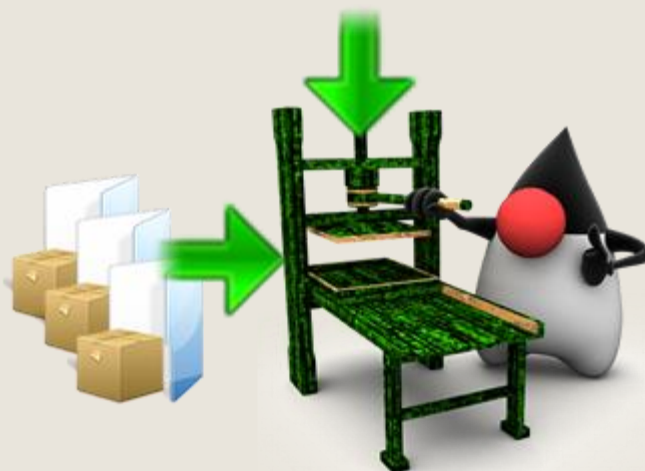
- $Reach(TS)$ היא קבוצת כל המצבים הנגישים במערכת TS

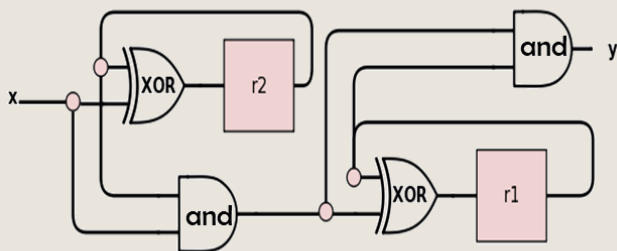
סוג מרכזי של תכונות:

קיום מצב נגיש שתיוגו מכיל צירוף מסוים של פסוקים אטומיים

בהרבה מקרים, לא נרצה לכתוב
מערכות מעברים באופן ישיר

נעדיף לתאר את המערכת בשפה "עילית"
ולהשתמש בקומפיילר שייצר מערכת מעברים





#reg = 2
#in = 1
#out = 1

$$\text{upd}(\langle r_1, r_2 \rangle, x) = \langle (x \wedge r_2) \oplus r_1, x \oplus r_2 \rangle$$

$$\text{out}(\langle r_1, r_2 \rangle, x) = \langle r_1 \wedge r_2 \wedge x \rangle$$

$$S = \{0,1\}^{\#reg} \times \{0,1\}^{\#in}$$

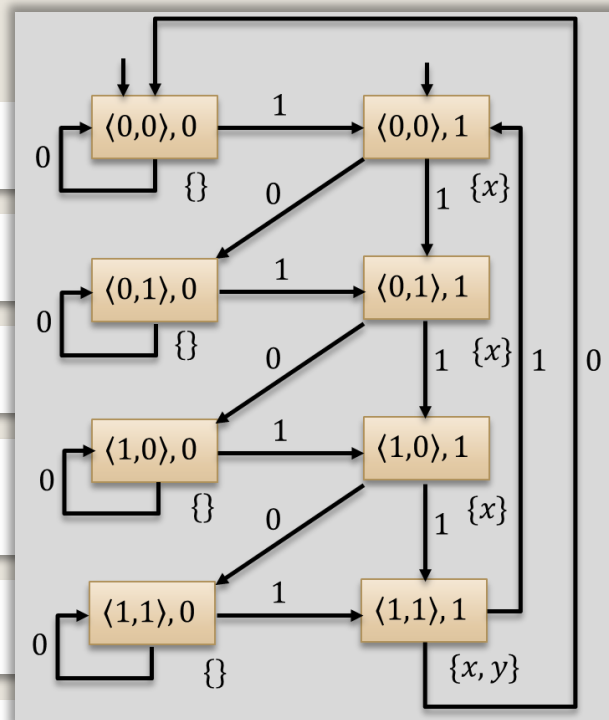
$$Act = \{0,1\}^{\#in}$$

$$I = \{0\}^{\#reg} \times \{0,1\}^{\#in}$$

$$\rightarrow = \{ \langle r, x \rangle \xrightarrow{x'} \langle \text{upd}(r, x), x' \rangle : \langle r, x \rangle \in S \}$$

$$AP = \{ r_1, \dots, r_{\#reg}, x_1, \dots, x_{\#in}, y_1, \dots, y_{\#out} \}$$

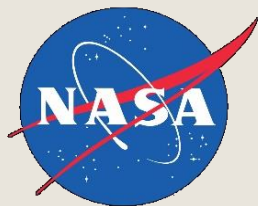
$$L(s = \langle \langle r_1, \dots, r_{\#reg} \rangle, \langle x_1, \dots, x_{\#in} \rangle \rangle) = \{ r_i : r_i = 1 \} \cup \{ x_i : x_i = 1 \} \cup \{ y_i : y_i = 1, \langle y_1, \dots, y_{\#out} \rangle = \text{out}(s) \}$$



למרות שהשימוש במערכות מעברים ובטכניקת
בדיקות-מודל הוכיח עצמו ונמצא בשימוש תעשייתי
בעיקר בתחום מעגלי החומרה



אנחנו נתרכז בקורס בשימושים בתחום התוכנה



לכן, נגדיר תרגום ממבנה שנקרא "גרף תוכנית"
(סוג של תרשים זרימה) למערכת מעברים



מעברים מִתְנִים במערכת מעברים

- תוכניות תלויות-נתונים בנויות מפקודות מהצורה:

```
if  $x \% 2 = 1$  then  $x := 3x + 1$  else  $x := x/2$  fi
```

ראו: https://he.wikipedia.org/wiki/השערת_קולץ

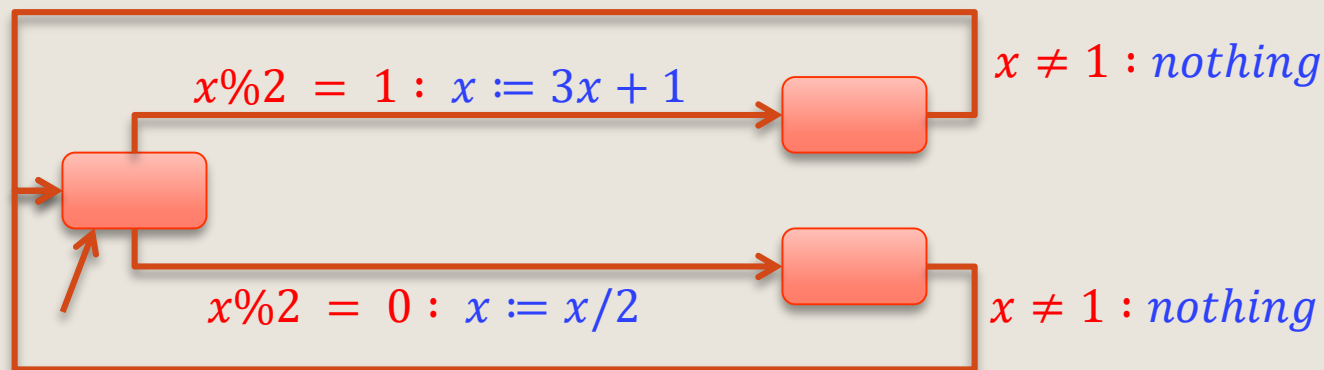
- אפשרויות למידול כמערכת מעברים:

- אפשר **להשמיט** את התנאים ולתרגם למעבר לא דטרמיניסטי

- אפשר להגדיר **שפה עילית נוספת** שתתורגם למערכת מעברים:



זאת לא
מערכת מעברים!



- נקרא לזה **גרף תוכנית (program graph)**

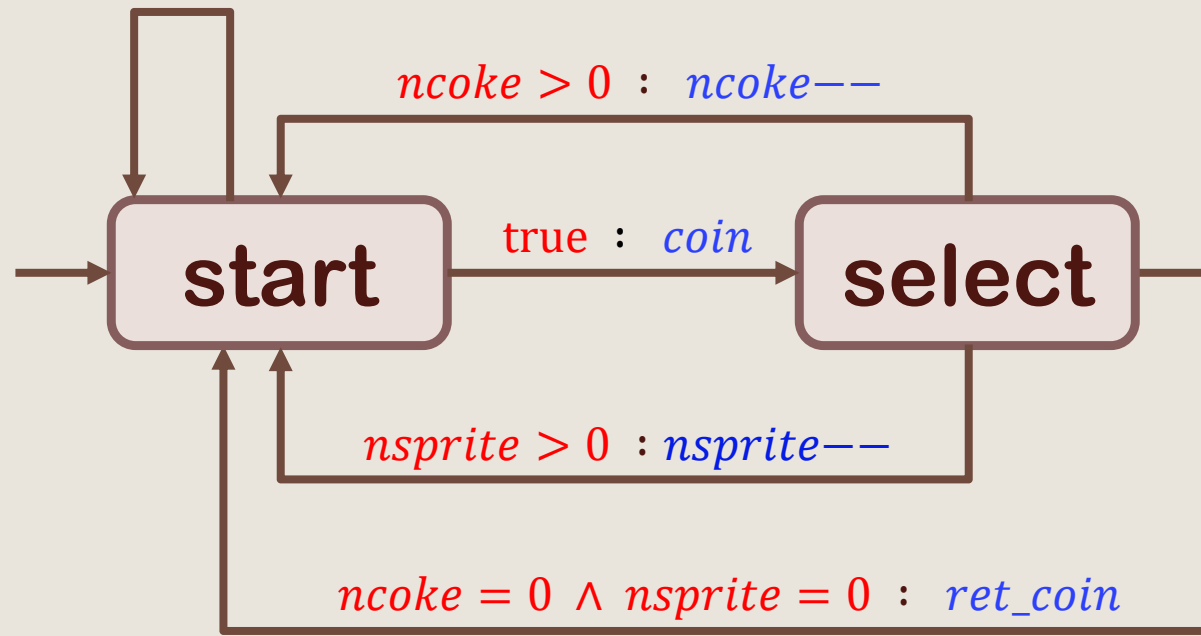
- אח"כ נראה איך פורסים (unfold) גרף-תוכנית למערכת מצבים (באופן אוטומטי)

דוגמה: שכלול מודל מכונת המשקאות

(אנחנו בונים כאן **גרף תוכנית**, לא מערכת מעברים)

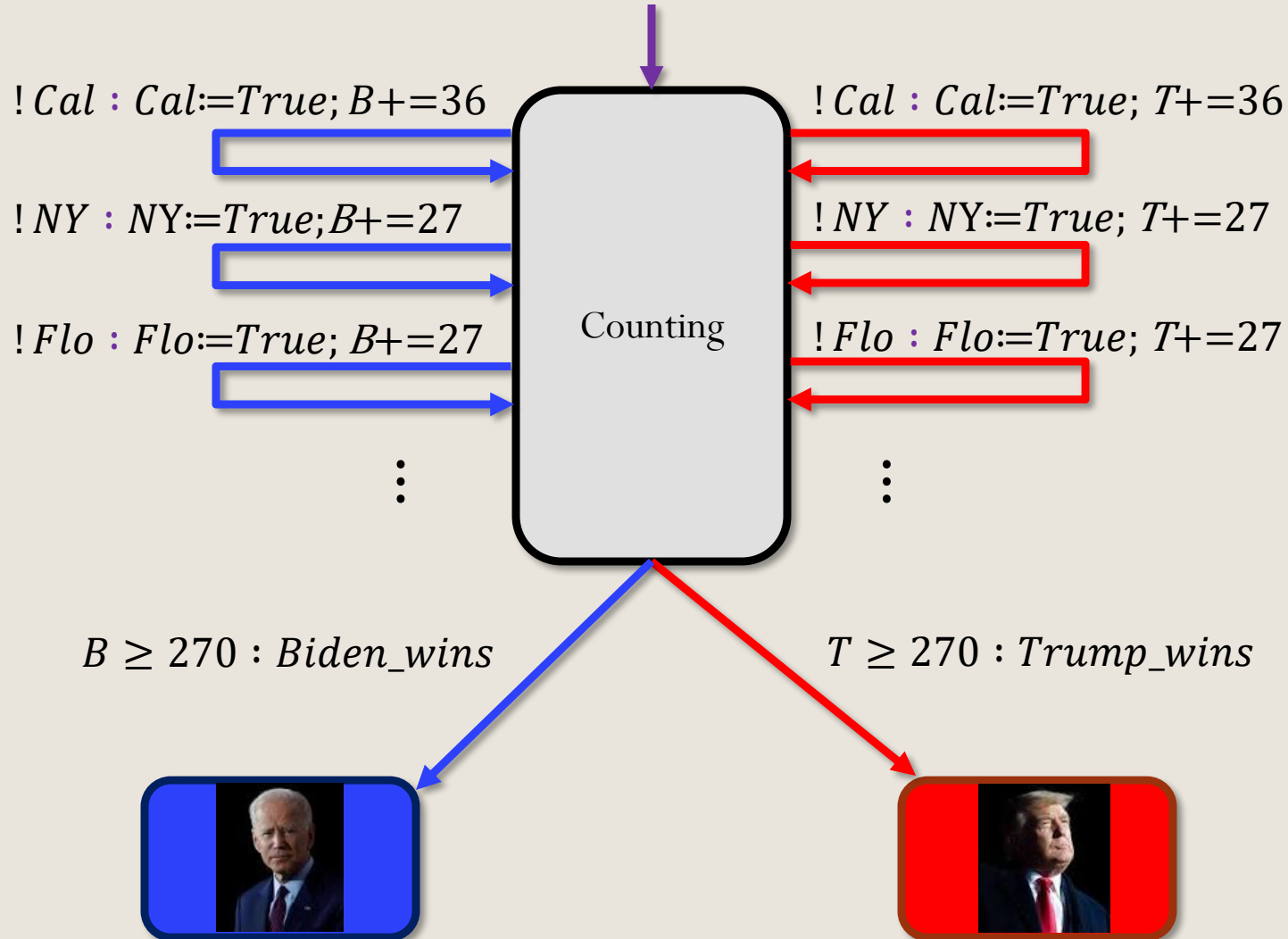
סופרים כמות בקבוקי הספרייט והקולה מטבע אם המכונה ריקה:

true : $ncoke = 10 \wedge nsprite = 10$



דוגמה: ספירת הקולות בארה"ב

$$!Cal \wedge !NY \wedge !Flo \cdots \wedge T = B = 0$$





חישוב $|Reach(TS)|$

```

reps = [1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 3, 3, 3, 4, 4, 4, 4, 4, 4, 5, 5, 5, 6,
        6, 7, 7, 7, 8, 8, 8, 8, 9, 9, 9, 9, 10, 11, 12, 13, 14, 14, 16, 18, 18, 27, 27, 36]

I = {(0, 0, tuple((False for i in range(49))))}

def markTrue(flags, i):
    return flags[:i] + (True,) + flags[i + 1:]

def post(s):
    return (set() if s[0] >= 270 or s[1] >= 270 else
            ((s[0]+reps[i], s[1], markTrue(s[2], i)) for i in range(49) if not s[2][i]) |
            {(s[0], s[1]+reps[i], markTrue(s[2], i)) for i in range(49) if not s[2][i]})

def POST(S):
    return set.union(*[post(s) for s in S])

S = I
for i in range(3):
    print(f'{len(S)=}')
    S = POST(S)

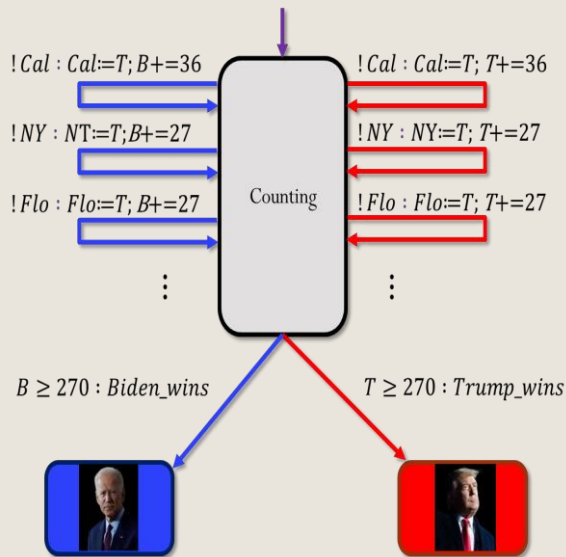
```

$\text{len}(S)=1$
 $\text{len}(S)=98$
 $\text{len}(S)=???$

1. $98 \cdot 48 \cdot 2$
2. $4 \cdot \binom{49}{2} - 99$
3. ...



הגדרות פורמליות: הצבות, תנאים והשפעה



• נקרא לקבוצת המשתנים שבתוכנית *Var*

• לקבוצת ההשמות (evaluations): *Eval(Var)*

• השמה היא פונקציה המתאימה לכל משתנה ערך.

• נתאר בד"כ כאוסף הזוגות: $\langle x \mapsto 3, y \mapsto 'Hi' \rangle$

• לקבוצת התנאים (conditions): *Cond(Var)*

• $\models: Cond(Var) \times Eval(Var) \rightarrow \{True, False\}$

• פונקציה הקובעת ערך אמת לכל תנאי ולכל השמה

• $\models (x < 9, \langle x \mapsto 3, y \mapsto 'Hi' \rangle) = True$

• $\langle x \mapsto 3, y \mapsto 'Hi' \rangle \models x < 9$

• $\langle x \mapsto 3, y \mapsto 'Hi' \rangle \not\models x < 2$

• לקבוצת הפעולות: *Act*

• *Effect*: $Act \times Eval(Var) \rightarrow Eval(Var)$

• פונקציה המתאימה לכל פעולה ולכל השמה את ההשמה שתהיה לאחר הפעולה.

• $Effect(x++, \langle x \mapsto 3, y \mapsto 'Hi' \rangle) = \langle x \mapsto 4, y \mapsto 'Hi' \rangle$

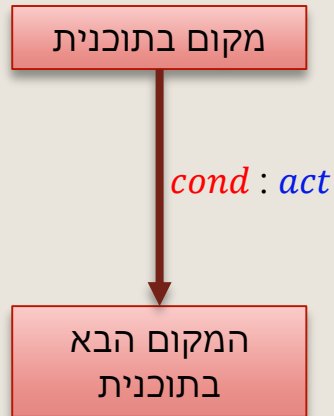


הגדרה: גרף תוכנית

גרף תוכנית (**program graph**) מעל קבוצת משתנים Var הוא שביעיה סדורה

$$\langle Loc, Act, \models, Effect, \rightarrow, Loc_0, g_0 \rangle$$

באשר:



• Loc היא קבוצת **המקומות** (locations)

• $Loc_0 \subseteq Loc$ היא קבוצת מקומות **ההתחלה**

• Act היא קבוצת **הפעולות**

• $\models: Cond(Var) \times Eval(Var) \rightarrow \{True, False\}$ היא **פונקציית הערכת התנאים**

• $Effect: Act \times Eval(Var) \rightarrow Eval(Var)$ היא **פונקציית ההשפעה**

• $\rightarrow \subseteq Loc \times Cond(Var) \times Act \times Loc$ הוא **יחס המעברים**

• $g_0: Loc_0 \rightarrow Cond(Var)$ הוא **תנאי ההתחלה**



גרף תוכנית למכונת משקאות המשוכללת

$Var = \{nsprite, ncoke\}$

$Loc = \{start, select\}$

$Act = \{get_coke, get_sprite, coin, ret_coin, refill\}$

$Effect(coin, \eta) = \eta$

$Effect(ret_coin, \eta) = \eta$

$Effect(refill, \eta) = [nsprite := 100, ncoke := 100]$

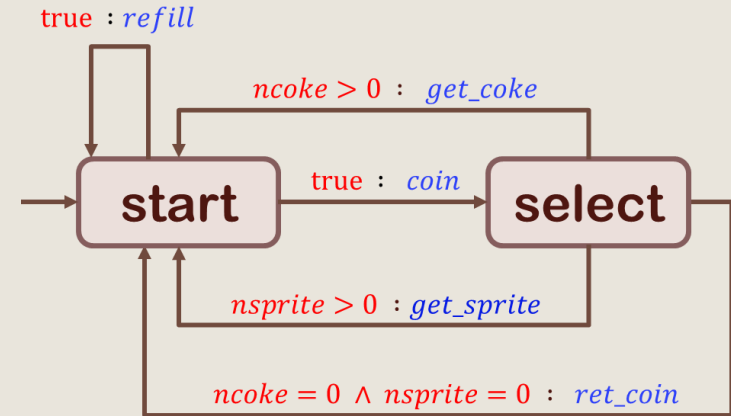
$Effect(get_sprite, \eta) = \eta[nsprite := nsprite - 1]$

$Effect(get_coke, \eta) = \eta[ncoke := ncoke - 1]$

$Loc_0 = \{start\}$

$g_0 = (nsprite = 100 \wedge ncoke = 100)$

סימון לבניית פונקציה חדשה הנבדלת
מאחרת רק בערך מסוים



יחס המעברים:

$start \xrightarrow{true : coin} select$

$start \xrightarrow{true : refill} start$

$select \xrightarrow{ncoke > 0 : get_coke} start$

$select \xrightarrow{nsprite > 0 : get_sprite} start$

$select \xrightarrow{ncoke = 0 \wedge nsprite = 0 : ret_coin} start$



```
VENDING_PG = {
  'Loc': {'start', 'select'},
  'Loc0': {'start'},
  'Act': {'coin', 'refill', 'get_coke', 'get_sprite', 'ret_coin'},
  'Eval': evaluate,
  'Effect': effect,
  'to': {
    ('start', 'true', 'coin', 'select'),
    ('start', 'true', 'refill', 'start'),
    ('select', 'ncoke > 0', 'get_coke', 'start'),
    ('select', 'nsprite > 0', 'get_sprite', 'start'),
    ('select', 'ncoke=0 && nsprite=0', 'ret_coin', 'start')
  },
  'g0': {'ncoke':10, 'nsprite':10}
}
```

קוד לבניית גרף התוכנית

פונקציית פירוש תנאים (E)

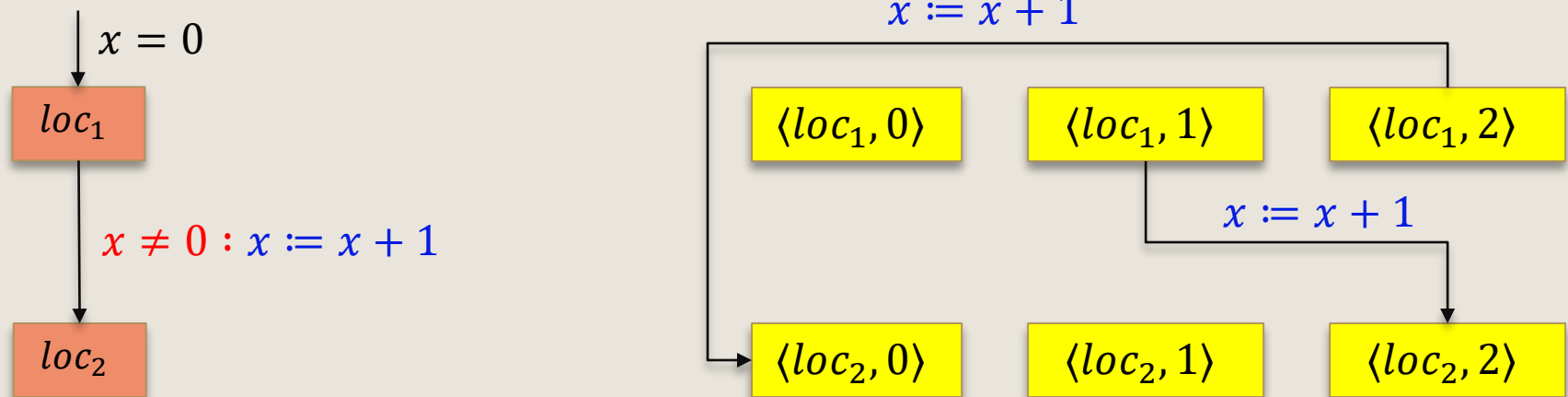
```
def evaluate( cond, eta ) :
  return {
    'true': lambda eta: True,
    'ncoke > 0': lambda eta: eta['ncoke']>0,
    'nsprite > 0': lambda eta: eta['nsprite']>0,
    'ncoke=0 && nsprite=0': lambda eta: eta['ncoke']==0 and eta['nsprite']==0
  }[cond](eta)
```

```
def effect( act, eta ) :
  return {
    'coin': lambda eta: eta,
    'ret_coin': lambda eta: eta,
    'refill': lambda eta: {'ncoke':10, 'nsprite':10},
    'get_coke': lambda eta: eta | {'ncoke': eta['ncoke']-1},
    'get_sprite': lambda eta: eta | {'nsprite': eta['nsprite']-1},
  }[act](eta)
```

פונקציית פירוש פעולות



תרגום אוטומטי מגרף תוכנית למערכת מעברים



PG

$TS(PG)$

נקרא לפעולה זאת פריסה



מגרף תוכנית למערכת מעברים

הגדרה פורמאלית של פעולת הפריסה

בהנתן גרף תוכנית PG , נגדיר מערכת מעברים $TS(PG)$:

• **מצבים:** $S = Loc \times Eval(Var)$

• **יחס מעברים:** $S \times Act \times S \rightarrow \subseteq$ מוגדר ע"י

$$\frac{l \xrightarrow{g:\alpha} l' \wedge \eta \models g}{\langle l, \eta \rangle \xrightarrow{\alpha} \langle l', Effect(\alpha, \eta) \rangle}$$

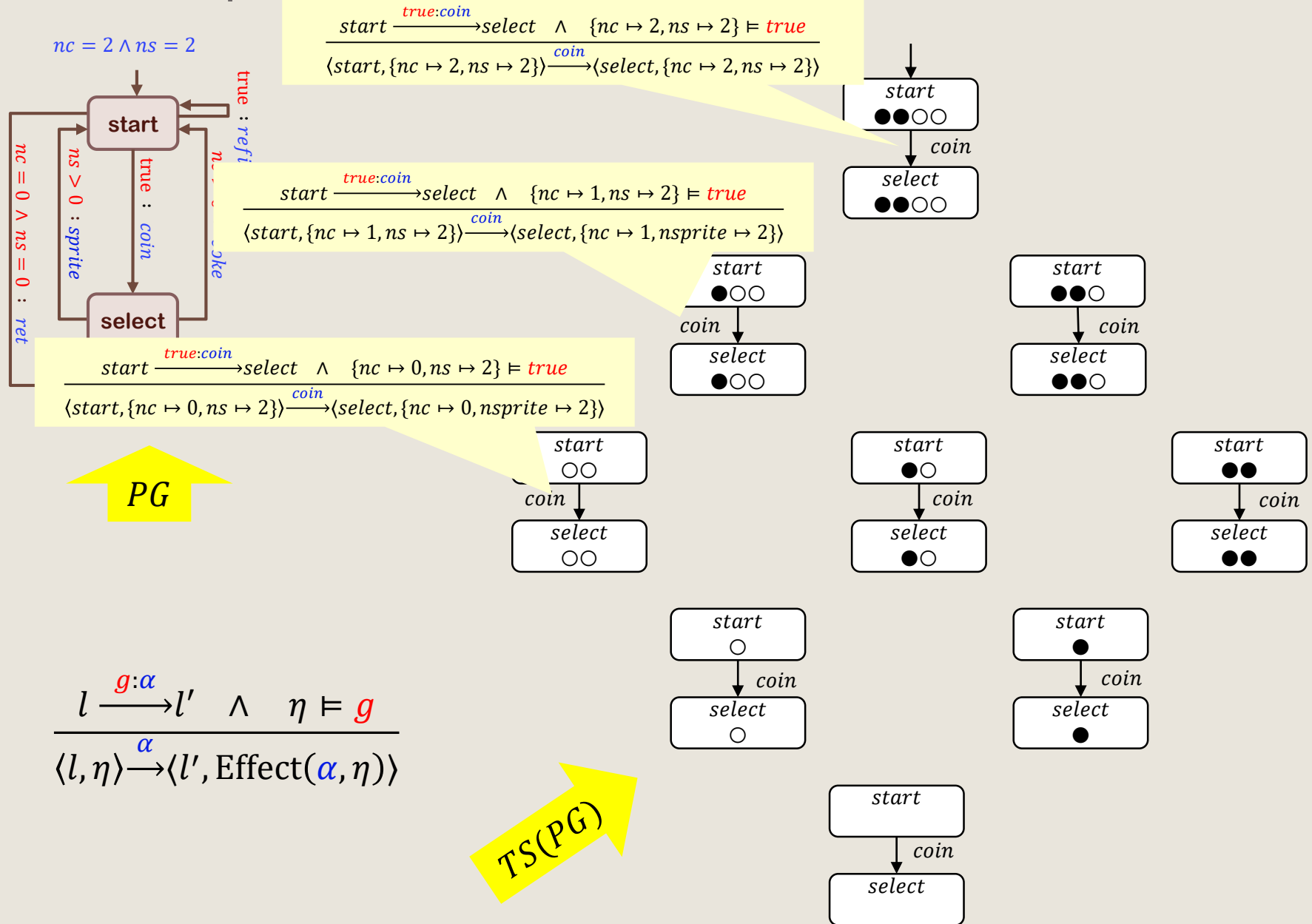
• **מצבי התחלה:** $I = \{ \langle l, \eta \rangle : l \in Loc_0, \eta \models g_0(l) \}$

• **פסוקים אטומיים:** $AP = Loc \cup Cond(Var)$

• **תיוג:** $L(\langle l, \eta \rangle) = \{l\} \cup \{g \in Cond(Var) : \eta \models g\}$

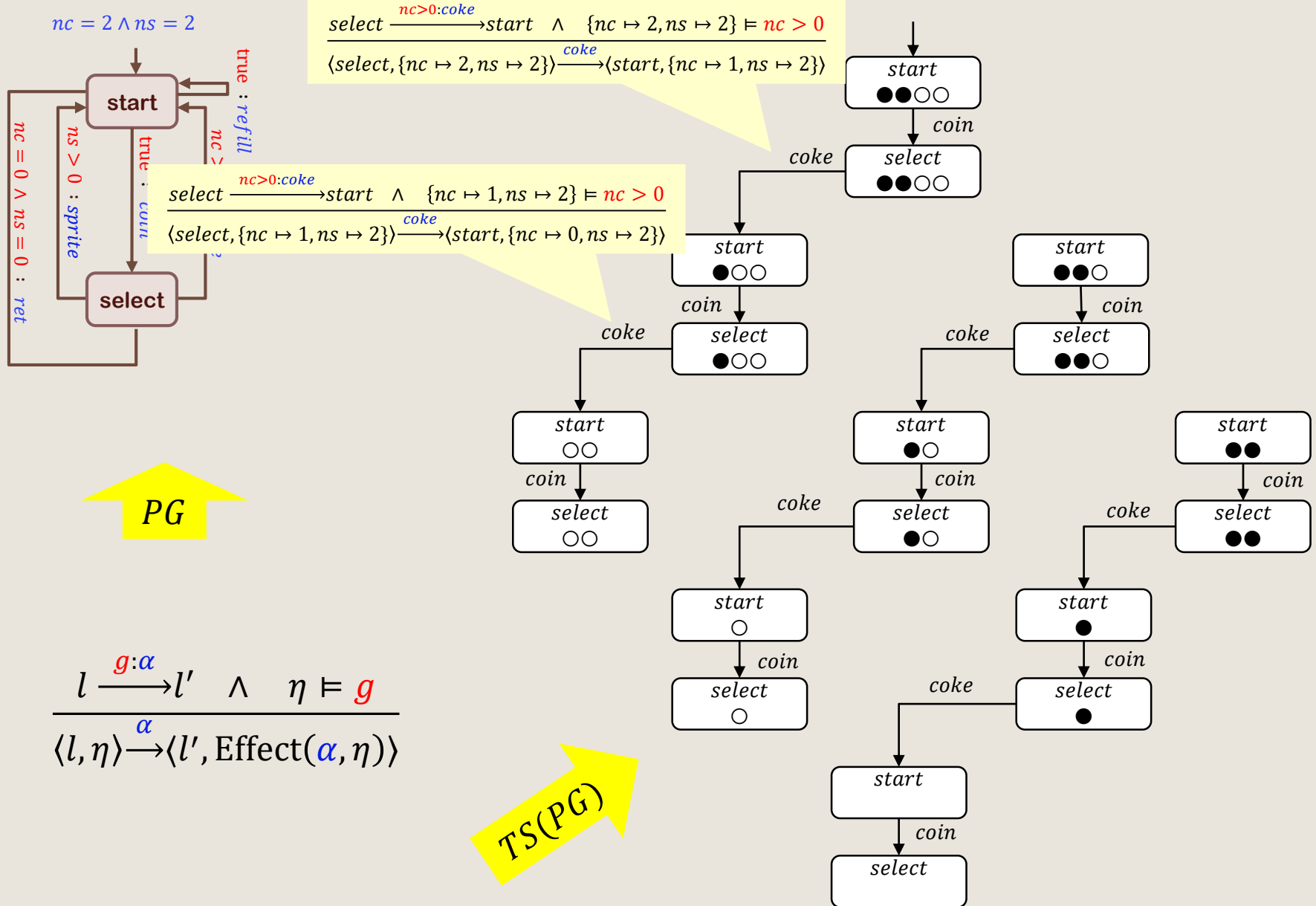


דוגמה: פריסת מערכת המעברים למכונת המשקאות



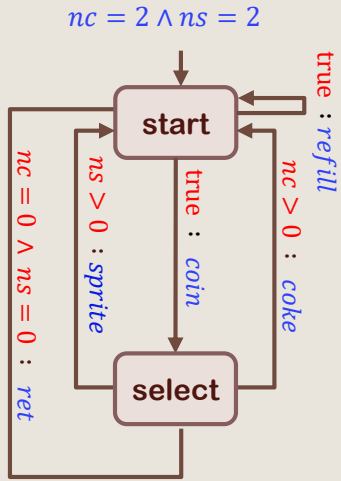


דוגמה: פריסת מערכת המעברים למכונת המשקאות





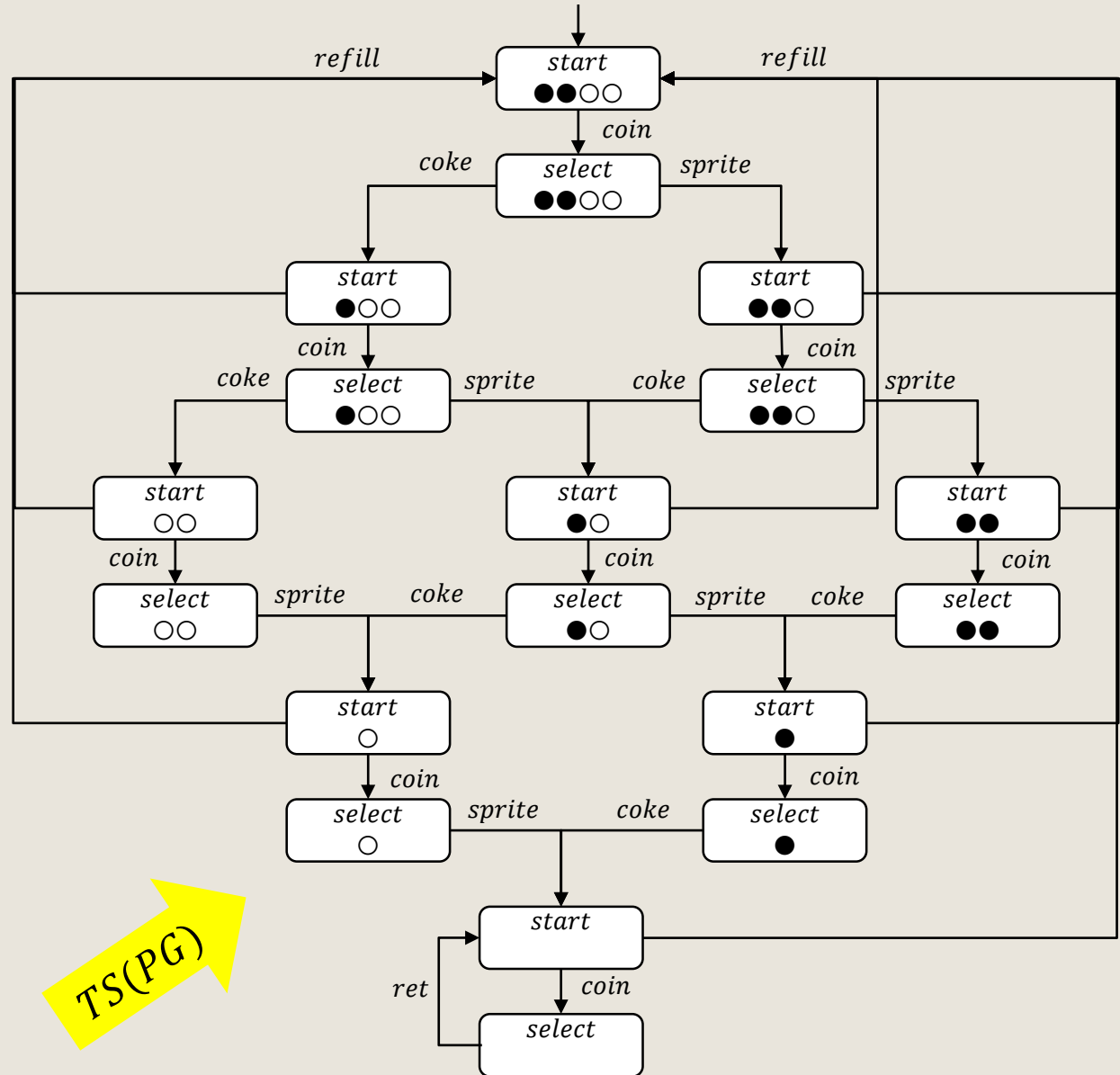
דוגמה: פריסת מערכת המעברים למכונת המשקאות



PG

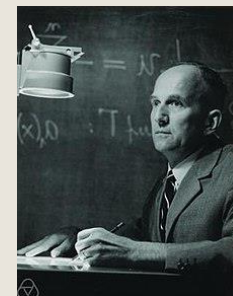
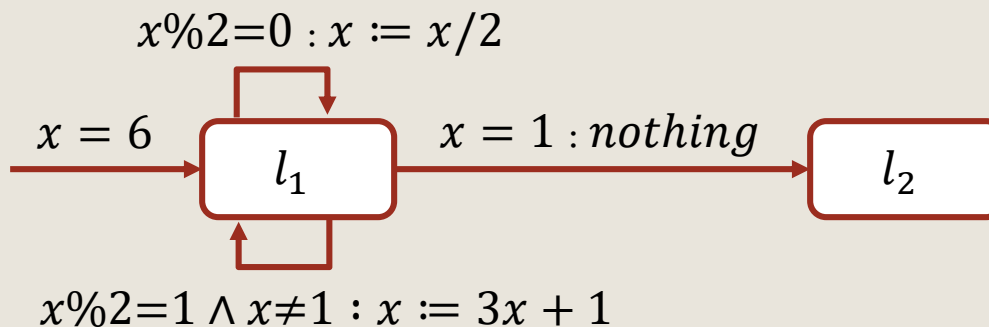
$$\frac{l \xrightarrow{g:\alpha} l' \quad \wedge \quad \eta \models g}{\langle l, \eta \rangle \xrightarrow{\alpha} \langle l', \text{Effect}(\alpha, \eta) \rangle}$$

TS(PG)

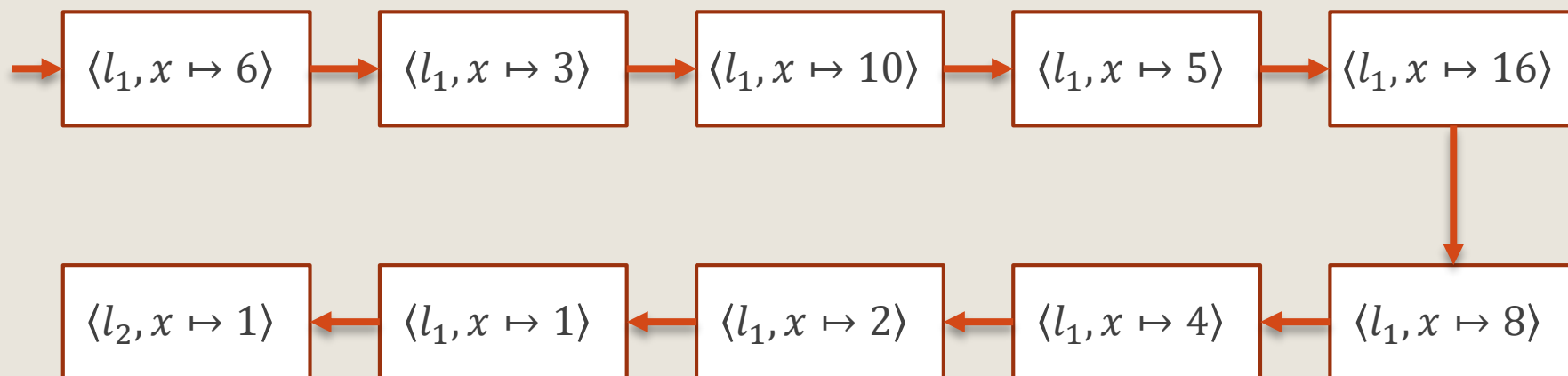


דוגמה: השערת קולץ (Collatz)

PG:



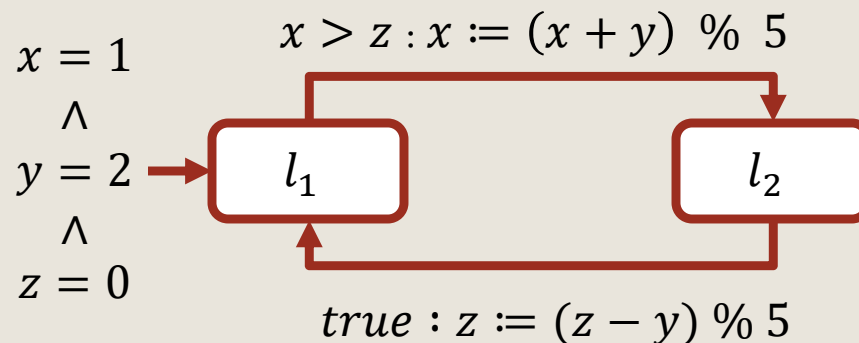
TS(PG):



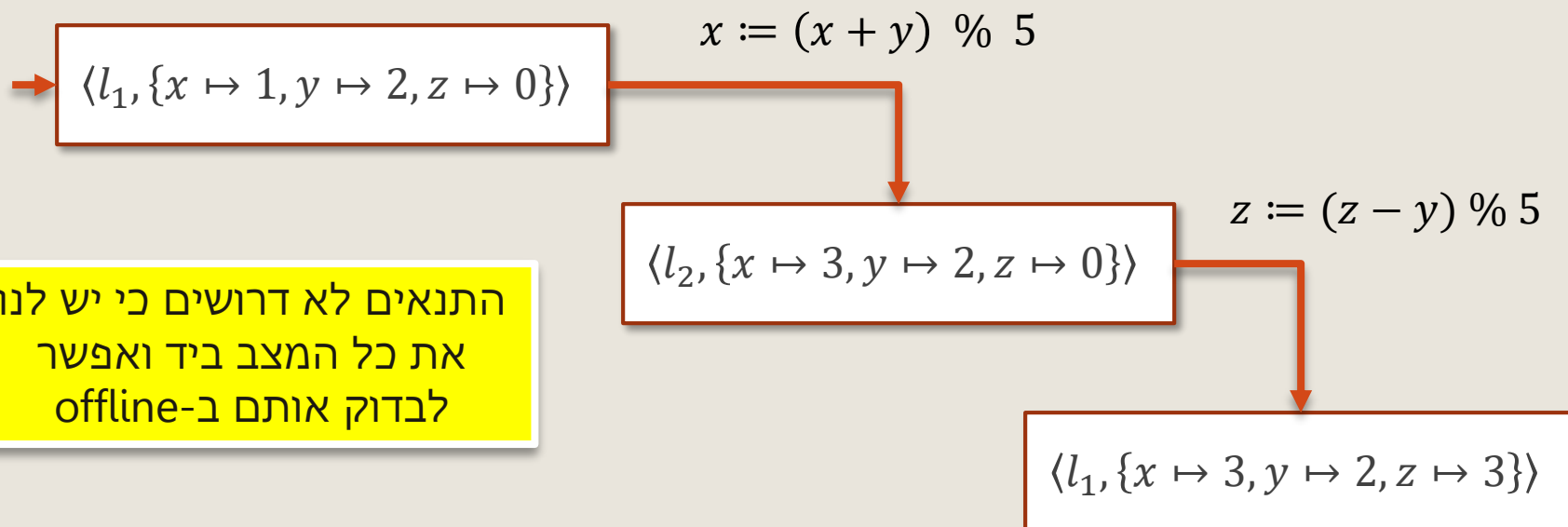


דוגמה: במערכת המעברים אין תנאים

PG:



TS(PG):



התנאים לא דרושים כי יש לנו
את כל המצב ביד ואפשר
לבדוק אותם ב-offline



סיכום

- גרף תוכנית הוא מודל המתאר את הקוד של התוכנית
- נתרגם גרף תוכנית PG למערכת מעברים $TS(PG)$ שאותה נבדוק
- מצב של מערכת המעברים $TS(PG)$ = מקום בתוכנית + ערכי המשתנים
- מעברים ב- $TS(PG)$ מתאימים לביצוע פקודה בתוכנית. ייתכן שישתנו ערכי המשתנים ו/או המקום הבא לביצוע
- ב- $TS(PG)$ **אין תנאים** כי אנחנו מקודדים באופן מפורש את כל המעברים האפשריים. אם התנאי בתוכנית לא מתקיים עבור ההשמה המסוימת שמתאר המצב, לא נמתח חץ.
- השלב הבא: **תוכניות עם ריבוי תהליכים (multi threaded)**